

Yes — **robots** is **actually a better name** if you already know you want the same software architecture to eventually support K9 as well as the ROV.

I would make that the **top-level repository**, and treat the individual robots as configurations/implementations within it.

Something like:

```
robots/
├─ apps/
│   ├── cockpit/
│   ├── control/
│   ├── telemetry/
│   ├── video/
│   └─ datalogger/
├─ robots/
│   ├── rovr/
│   │   ├── config/
│   │   ├── control/
│   │   └─ hardware/
│   └─ k9/
│       ├── config/
│       ├── control/
│       └─ hardware/
├─ packages/
│   ├── messages/
│   ├── nats/
│   ├── hardware/
│   └─ common/
├─ frontend/
├─ configs/
├─ tests/
├─ scripts/
└─ docs/
```

However, I would **not necessarily put** `rovr/` **and** `k9/` **in there immediately.**

There's an important architectural distinction:

Generic software

Things that should work for any robot:

```
apps/  
packages/
```

For example:

- Cockpit
- NATS
- WebSocket telemetry
- video
- logging
- authentication
- configuration
- generic telemetry
- generic command handling

Robot-specific behaviour

Things that genuinely depend on the vehicle:

```
robots/  
├─ rovr/  
└─ k9/
```

For example, the ROV might have:

```
depth  
heading  
thruster allocation  
buoyancy  
underwater cameras
```

while K9 might eventually have:

```
wheel/leg control  
head movement  
arm control  
different camera arrangements
```

There's an even cleaner possibility

Given what you've already built, I'd consider making the **robot definition primarily configuration**, rather than separate code.

For example:

```
configs/  
├─ robots/  
│   ├─ rov.yaml  
│   └─ k9.yaml
```

Then:

YAML

```
robot:  
  id: rov  
  name: ROV  
  type: underwater
```

capabilities:

- drive
- depth
- heading
- camera
- lights

and:

YAML

```
robot:  
  id: k9  
  name: K9  
  type: ground
```

capabilities:

- drive
- camera
- head
- lights

The cockpit can then discover:

"I'm connected to a robot with these capabilities."

and render the appropriate UI.

That's particularly compatible with your existing **robot profile** concept in the cockpit.

And robots has another advantage

It gives you a very clean relationship between the two repositories:

```
robots/  
├──  
│   ├── actual robot software  
│   ├──  
│   ├── R0V configuration  
│   └──  
│       └── K9 configuration
```

```
robots-hil/  
├──  
│   ├── R0S 2  
│   ├── Gazebo  
│   ├── simulated R0V  
│   └── simulated K9 (eventually)
```

Or, if you want the HIL repository name to match:

```
robots  
robots-hil
```

I quite like that.

It communicates that **the first repository is the real robot software**, while the second is the environment used to test it.

One thing I would *not* do

I wouldn't make this:

```
robots/  
├── rov/  
│   └── entire application  
└── k9/  
    └── entire application
```

because you'd quickly end up duplicating:

```
cockpit
NATS
video
logging
telemetry
authentication
configuration
```

and then maintaining two versions.

Instead:

```
robots/
├─ common software
├─ robot-specific software
└─ robot configurations
```

That gives you **one cockpit, one communications architecture, one video system**, etc., while allowing the actual vehicles to differ where they need to.

So, yes: **I'd be happy with `robots` as the repository name**, and I think it's a better long-term choice than `rov`.

Yaml vs json vs something else?

For the `robots` project, I would choose **YAML for human-edited configuration, JSON for machine/API data, and a typed schema for the actual contract**.

My recommendation

Use	Format	Why
Robot profiles	YAML	Easy for humans to read/edit
Deployment/ configuration	YAML	Good for structured configuration
NATS messages	JSON initially	Easy to inspect/debug
REST/WebSocket data	JSON	Natural fit for web tooling
Persistent telemetry/logs	SQLite	Don't turn configuration files into a database

Use	Format	Why
Configuration validation	JSON Schema / Pydantic	Gives us an actual contract

For example, your robot profile could be:

YAML

```
id: rovr
name: ROV
type: underwater

capabilities:
  - drive
  - depth
  - heading
  - attitude
  - camera
  - lights

interfaces:
  control: nats
  hardware: rs485

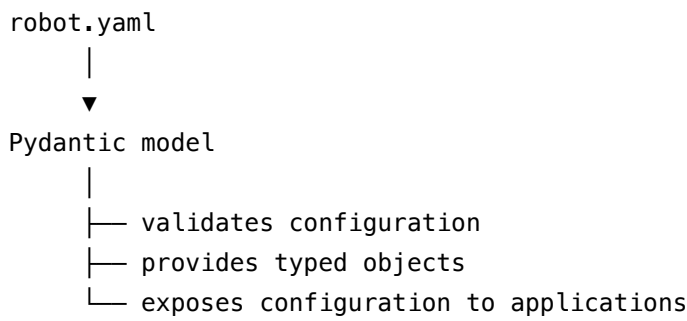
cameras:
  - id: forward
    type: usb
  - id: panoramic
    type: kogeto
    dewarp: true
```

That is substantially nicer to maintain than equivalent JSON.

But I would not let YAML define your software interface

This is the important distinction.

I'd have something like:



So the application doesn't do this everywhere:

Python

Run

```
config["capabilities"][3]
```

It does something more like:

Python

Run

```
robot.capabilities
```

That gives you compile-/run-time structure without sacrificing the convenience of YAML.

For NATS

I'd initially use **JSON messages**:

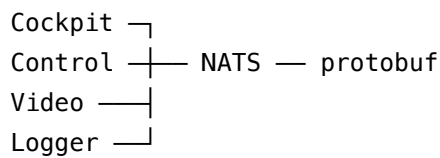
JSON

```
{
  "robot_id": "rov",
  "timestamp": "2026-08-22T22:00:00Z",
  "depth_m": 12.4,
  "heading_deg": 183.2
}
```

But there's an important caveat.

Don't make JSON the permanent wire protocol if performance becomes important.

Because you're eventually talking about high-rate telemetry and control, we could move the NATS payloads to something schema-based such as **Protocol Buffers** later:



That gives you:

- explicit types
- versioned schemas
- smaller messages
- faster serialisation
- language independence
- much less ambiguity about units and field types

For your current stage, however, **JSON is much easier to debug**. I'd rather get the architecture right first than introduce protobuf prematurely.

So I'd settle on

```

robots/
|
|— configs/
|   |
|   |— robots/
|   |   |
|   |   |— rov.yaml
|   |   |— k9.yaml
|   |
|   |— packages/
|   |   |
|   |   |— messages/
|   |   |   |— schemas/
|   |
|   |— apps/
|   |   |
|   |   |— cockpit/
|   |   |— control/
|   |   |— telemetry/
|   |   |— video/
|   |   |— datalogger/
|   |
|   |— ...
|
|— ...
  
```

YAML = configuration.

JSON = human-readable data exchange.

Pydantic/schema = the contract.

SQLite = persistent structured data.

Protobuf = a future option for high-rate NATS traffic.

For the `robots` project, I think that's a very good balance between engineering discipline and not over-engineering the system before we need to.

What about toml?

Yes — **TOML is a serious contender**, and for your `robots` repository I would actually use **TOML in some places**.

The key is that YAML, TOML, and JSON have different strengths.

For your project

Requirement	My choice
Python project metadata	TOML
Application configuration	TOML or YAML
Robot profile	TOML
NATS message payload	JSON initially
API/WebSocket payload	JSON
Persistent data	SQLite
Typed contract	Pydantic/schema

The big advantage of TOML is that it sits nicely between JSON and YAML: **human-editable, structured, but considerably less ambiguous than YAML**.

For example, your ROV configuration could be:

TOML

```
[robot]
id = "rov"
name = "ROV"
type = "underwater"

[capabilities]
drive = true
depth = true
heading = true
attitude = true
camera = true
lights = true

[interfaces]
control = "nats"
hardware = "rs485"

[[cameras]]
id = "forward"
type = "usb"

[[cameras]]
id = "panoramic"
type = "kogeto"
dewarp = true
```

I actually prefer that to the YAML equivalent for **engineering configuration**.

Why I'd favour TOML for robots

You're an engineer, and this project is going to contain a lot of things like:

```
RS-485 addresses
baud rates
timeouts
control-loop frequencies
limits
camera parameters
NATS subjects
ports
GPIO assignments
units
hardware identifiers
```

TOML makes those explicit without YAML's more surprising parsing behaviour.

For example:

TOML

```
[rs485]
port = "/dev/ttyUSB0"
baudrate = 115200
timeout_ms = 20

[control]
loop_rate_hz = 100
command_timeout_ms = 250

[limits]
max_depth_m = 100.0
max_horizontal_speed_m_s = 1.5
```

That's very readable.

TOML also fits Python particularly well

Your Python applications can use Python's built-in `tomllib` for reading TOML, so you don't need another dependency just to parse the configuration.

And your existing `pyproject.toml` already puts TOML in the Python ecosystem you're using.

One caveat

TOML isn't particularly good for **deeply hierarchical, highly dynamic data**.

I wouldn't try to make NATS telemetry look like:

```
TOML

[telemetry]
...
```

TOML is a **configuration format**, not a streaming data format.

Likewise, if you eventually have complex robot descriptions with hundreds of nested parameters, YAML may become more convenient.

Therefore I'd revise my previous recommendation slightly

For `robots`, I'd go with:

TOML

- |— robot configuration
- |— hardware configuration
- |— application configuration
- └— deployment configuration

JSON

- |— NATS messages initially
- |— WebSocket messages
- └— REST API

SQLite

- └— persistent telemetry / logs

Pydantic

- └— validates configuration and defines application models

And I'd make **TOML the default configuration format** unless we encounter a specific configuration that YAML handles substantially better.

In fact, given the direction we're taking with the project, **I'd choose TOML over YAML for the robot profiles we were discussing.**

ChatGPT can make mistakes. Check important info.